



SyncVerse

Collaborative Real-Time Code Editor
for Remote STEM Education

Master Build Blueprint

Scope | System architecture | Every feature | Work plan for four people

A Lane A - Platform, Collaboration and Comms

B Lane B - Execution and Visualization

C Lane C - AI Tutor and Code Quality

D Lane D - Learning, Classroom and Experience

Team

Akshit Sharma | Miti Shah | Simrit Kaur | Rahil Shah

Problem statement PS 02 | Collaborative Real-Time Code Editor

Version 1.0 | 30 September 2026 | DRAFT FOR TEAM REVIEW - nothing has been built yet

Built from: the PS 02 problem statement and the PS 02 Final Combined Feature Compendium.

Contents

1. Executive summary	4
How the seven PS requirements are covered	4
Six decisions to make first (full list in section 13)	4
Where the effort goes	5
How to read this document	5
2. Product definition	6
2.1 The problem in short	6
2.2 Who it serves	6
2.3 Design principles that settle arguments	6
2.4 Scope character	6
3. Feature catalog and tiers	7
Deferred to the roadmap (P3 - described, not built)	8
4. System architecture	9
4.1 Big picture	9
4.2 Components	9
4.3 Trust boundaries	10
4.4 Technology choices	10
4.5 Repository layout	11
5. Core flows	12
5.1 Private code run (lane B)	12
5.2 AI explanation and safe patch (lanes C, A)	12
5.3 Permission-gated debugging (lane D)	13
5.4 Trace, shared playback, predict and reflect (lanes B, D)	13
6. Feature deep dives	14
6.1 Real-time collaboration (lane A)	14
6.2 Sandboxed execution (lane B)	14
6.3 Python tracer and execution canvas (lane B)	14
6.4 AI tutor (lane C)	15
6.5 Code quality analysis (lane C)	16
6.6 Permission-gated debugging (lane D)	16
6.7 Predict and reflect (lane D)	17
6.8 Progress tracking and analytics (lane D)	17
7. Data model, permissions and API	18

7.1 Tables (Supabase Postgres)	18
7.2 Who can do what	18
7.3 API routes	19
7.4 Realtime channels	19
8. UX blueprint	20
Accessibility and reach (baseline, from the compendium)	20
9. How the team works	21
9.1 Your question: divide the work, or keep a sheet and take turns?	21
9.2 Lanes	21
9.3 Rules everyone follows	22
9.4 The tracker	22
9.5 Who depends on whom	22
9.6 Fallback: only one machine or one account	22
10. Phased plan and backlog	23
10.1 Timeline by phase and lane	23
10.2 Milestones	23
10.3 If time runs short: what to cut, in order	24
10.4 Backlog by lane	24
11. Risks and mitigations	29
12. Demo and submission plan	30
12.1 Golden path (about 7 minutes)	30
12.2 Planted-bug programs (built in T-B-05, used by T-C-10 and T-D-07)	30
12.3 Fallbacks	30
12.4 Submission checklist	30
13. Open decisions	31
13.1 Decisions	31
Appendix A. Environment variables	32
Appendix B. Judge0 reference	32
Appendix C. Concept taxonomy	33
Appendix D. Definition of done	33
Appendix E. Glossary	33

1. Executive summary

SyncVerse is a browser-based classroom for learning to code together. Learners and mentors edit one program live, talk on video inside the same tab, run code in a sandbox, and get an AI tutor that explains errors in plain language. Two ideas separate it from 'a shared document plus a compiler': **every learner runs and debugs on their own**, and the platform **checks each learner's understanding individually** instead of assuming that shared code means shared understanding.

RECOMMENDATION

Work as **four parallel lanes** (A Platform, B Execution, C AI and Quality, D Learning and Experience), each lane owned by one person, coordinated by a task tracker that lives in git and is updated by every teammate's Claude session. Do **not** run a strict 2-hour relay unless you are forced to share one machine - the backlog is already cut into 2-hour task cards, so it works in either mode. Details in section 9.

Product	Shared editor, video, AI tutor, sandboxed run, quality feedback, permissioned debugging and progress tracking in one tab.
Signature ideas	Shared code / private execution Execution canvas Predict-and-reflect checkpoints Permission-gated debugging AI patch preview with Accept / Reject
Architecture	Next.js on Vercel Yjs WebSocket service on Render Supabase (Postgres, Auth, Realtime) LiveKit Cloud Judge0 sandbox LLM API
Languages	Run: Python, C, C++, Java, JavaScript. Visualize (canvas): Python only.
Backlog	55 tasks 84.5 person-blocks (1 block = about 2 focused hours) = about 169 person-hours, split over 4 lanes plus shared work
Scope tiers	P0 must ship: 50.5 blocks (covers all 7 PS bullets) P1 differentiators: 26.0 P2 polish: 8.0 P3 roadmap only, not built
Per person	P0 only: about 25 h each P0 + P1: about 38 h each everything: about 42 h each (first-cut estimates, plus or minus 40%, assuming AI-assisted coding)

WATCH OUT

The full backlog is bigger than a typical hackathon weekend. That is deliberate: **P0 alone is a complete answer to the problem statement**, and every P1 and P2 task can be dropped without breaking another one (cut order in section 10.3). Decision D1 - how many hours you really have - picks the cut line.

How the seven PS requirements are covered

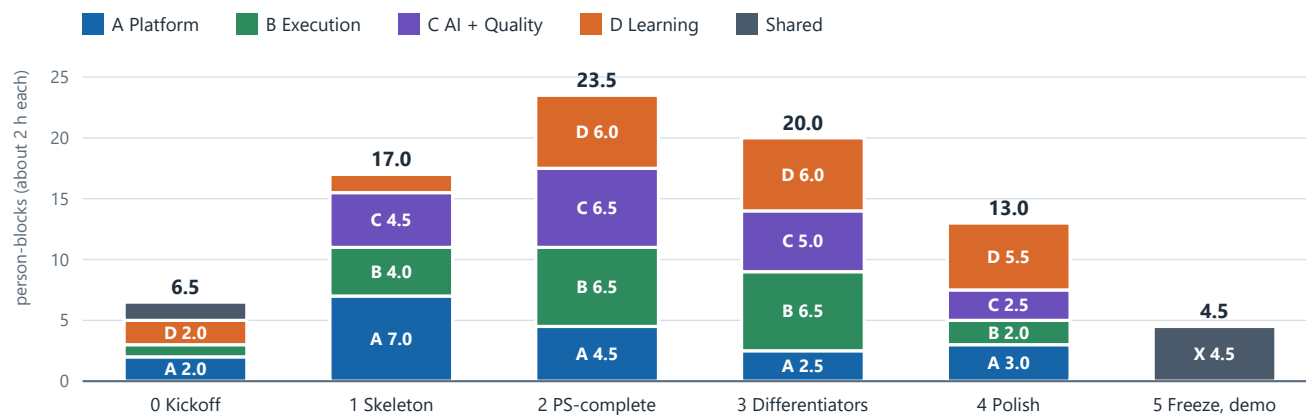
#	PS requirement	How SyncVerse delivers it	Tier
1	Real-time multi-user editing	Monaco + Yjs CRDT, labelled cursors, presence, persistent files	P0
2	Integrated video chat	LiveKit panel inside the workspace: video, audio, screen share, chat	P0
3	AI that explains syntax errors to beginners	LLM gateway with automatic context, structured explanation, patch preview; hint mode on top	P0 + P1
4	Live sandboxed code execution	Judge0, one independent job per learner, private results	P0
5	Automated code quality analysis	Analyzer adapters per language, quality panel, maintainability summary	P0
6	Collaborative debugging	Permission-gated access (P0) plus execution canvas and shared playback (P1)	P0 + P1
7	Personalized progress tracking	Learning events, student view (P0); class trends and predict-and-reflect evidence (P1)	P0 + P1

Six decisions to make first (full list in section 13)

- **D1** How many hours does each person really have, and when is the demo? This picks the cut line.
- **D2** Hosted Judge0 or a small self-hosted VM? Hosted is faster to start; quotas are the risk.
- **D3** Which LLM provider and who pays for the key? Set a hard spending cap.
- **D4** Guest sign-in for judges and demo (recommended) or accounts only?

- **D8** Who owns which lane? Suggested profiles are in section 9.2.
- **D12** Confirm the recommended tech stack (section 4.4). It is my proposal; the compendium does not prescribe products.

Where the effort goes



Person-blocks per phase, stacked by lane. Segment labels give lane and blocks, so colour is never the only cue. The same numbers as a table are in section 10.1.

How to read this document

Sections 2-5 say **what** we build and **how it fits together**. Sections 6-8 are the detailed designs. Sections 9-10 say **who does what and in what order**. Sections 11-13 cover risks, the demo and open decisions. Appendices hold reference tables. Task IDs such as T-B-03 are used everywhere and will be the IDs in the tracker.

2. Product definition

2.1 The problem in short

A remote coding lesson is stitched together from five or six unconnected tools: an editor, an online compiler, a video call, a chat app and, increasingly, an AI chatbot tab. A screen share shows what a student typed, not what the student misunderstood.

- **Tool fragmentation.** Every switch between tools costs time, context and the mentor's view of where a learner is stuck.
- **Error understanding gap.** Raw compiler and runtime messages are hard for beginners to turn into 'what actually happened and why'.
- **Shared code is not shared understanding.** In pair work one learner fixes the bug and the other copies it without knowing why it worked.

2.2 Who it serves

Persona	What they need	What SyncVerse gives them
Beginner student	Write, run, and understand failures without leaving the tab	Instant run, plain-English error explanations, tiered hints, patch preview they control
Intermediate / advanced student	Pair-program without stepping on each other's output; feedback on quality	Private runs per learner, quality panel, execution canvas
Mentor / teacher / TA	See who is stuck without demanding a screen share; help when invited; see class-wide gaps	Status-only overview, request-access flow, shared playback, class trends
Learning partner (peer)	Compare predictions and debug as equals	Shared canvas, predict and reveal, peer debug requests

2.3 Design principles that settle arguments

Principle	What it means when we build
1. Shared code, private execution	The source is collaborative. Stdin, output, run history and debug state belong to one learner unless they share them.
2. The student stays in control	AI changes are previews that need Accept. Debug access needs Allow and can be revoked. Nothing silently changes their work.
3. Explain the why, not only the fix	Explanations state cause and reasoning; hint mode asks a question instead of giving the answer.
4. One workspace	If a feature forces a tab switch, redesign it. Video, AI, run and canvas sit around the editor.
5. Observations, not scores	Progress is shown as evidence ('2 of 3 loop-boundary checks passed; retry recommended'). No grades, ranks or leaderboards.
6. Safe by default	Code only runs in the sandbox. Permissions are enforced on the server and in the database. Keys never reach the browser.
7. A learning environment, not a judge	No contest scoring, no hidden test cases, no plagiarism policing in this scope.

2.4 Scope character

Everything the compendium lists is triaged in section 3 into four tiers. **P0** is the literal problem statement at working quality. **P1** is what makes the project memorable to judges. **P2** is polish we take if time allows. **P3** is roadmap we describe but do not build. The compendium lists the execution canvas and predict-next-step checkpoints as signature differentiators (section 3), so they are P1, not P3.

3. Feature catalog and tiers

Every feature from the compendium, with its tier and the tasks that deliver it. Use this table when the team decides what to cut: find the feature, see the tasks, see the cost in section 10.4.

Feature	Tier	Tasks
2.1 Editor		
Real-time multi-user editing with conflict-free merge (CRDT)	P0	A-03 A-04
Live cursors and selections labelled with names	P0	A-05
Presence: online, typing, idle, active file	P0	A-05
File tabs, syntax highlighting, line numbers, auto-indent, language modes	P0	A-04 A-06
New file, upload, automatic language detection	P0	A-06
Persistent workspace (survives refresh, reconnect, service restart)	P0	A-03 A-12 00-04
Accounts (Google / email / guest), rooms, join by code, workspace roles	P0	A-01 A-02
Snapshots, version history and diff view	P2	A-10
2.2 Video		
Embedded video and audio with camera/mic controls, multi-participant	P0	A-07
Screen sharing	P0	A-07
In-room chat with code-snippet sharing	P0	D-12
Low-bandwidth mode	P2	A-11
2.3 AI		
Automatic-context error explanation (what / where / why / fix / snippet)	P0	C-01 C-02
Fault line highlighted in the editor	P0	B-04 C-02
Patch preview with explicit Accept / Reject; never silently applied	P0	C-03
Socratic hint mode (tiered hints)	P1	C-06
Explain code (selection, function, line-by-line)	P1	C-07
AI code review (logic and structure, with reasons)	P1	C-08
Recurring-error and misconception detection, concept tagging	P1	C-09
Explanation quality evaluation set and pre-baked demo answers	P1	C-10
Test-case and edge-case suggestions, complexity explanation	P2	C-11
Practice problems and micro-lessons from weak concepts	P2	C-12
Ghost Driver: spoken 'explain this step'	P2	B-10
2.4 Execution		
Sandboxed run with stdin, time and memory limits	P0	B-01 B-02
Independent per-user execution (shared code, private results)	P0	B-03
Normalized status categories with Explain-with-AI action	P0	B-01 B-02 B-04
Python, C, C++, Java, JavaScript; per-user run history	P0	B-05 B-02
2.5 Quality		
In-editor static analysis (formatting, naming, smells, complexity, security hints)	P0	C-04 C-05
Maintainability summary (not a grade) and Explain-this-finding	P0	C-05
2.6 Debugging		
Private-by-default execution and debugging	P0	B-03
Permission-gated access: request, allow, deny, revoke, expire	P0	D-02

Feature	Tier	Tasks
Grantee actions: view console, highlight line, propose edit, re-run	P0	D-02 C-03
Execution canvas (variables, arrays, call frames, step-through replay)	P1	B-06 B-07
Shared focus / follow-my-debugging (one presenter)	P1	B-08
Canvas accessibility equivalents	P2	B-09
2.7 Progress		
Learning event capture (runs, errors, AI help, checkpoints)	P0	D-03
Student progress view with observations and next-concept suggestions	P0	D-04
Mentor activity overview (status only, stuck flags)	P1	A-09
Class concept trends for mentors	P1	D-05
Stuck-detection nudge	P2	D-09
AI session summary	P2	D-10
3.3 Signature		
Predict-next-step checkpoints with individual follow-ups	P1	D-06
Experience		
Design system and app shell	P0	D-01
Landing page, guest onboarding, first-run tour	P1	D-08
Accessibility and UX pass	P1	D-11
Delivery		
Repo, CI, contracts, schema, smoke test, accounts	P0	00-01 00-02 00-03 00-05
Demo seed data and reset	P0	D-07
Golden-path tests, freeze, rehearsals, backup video, submission pack	P0	X-01 X-02 X-03 X-04 X-05

Task IDs are shown without the T- prefix (A-03 means T-A-03; 00-02 means T-00-02).

Deferred to the roadmap (P3 - described, not built)

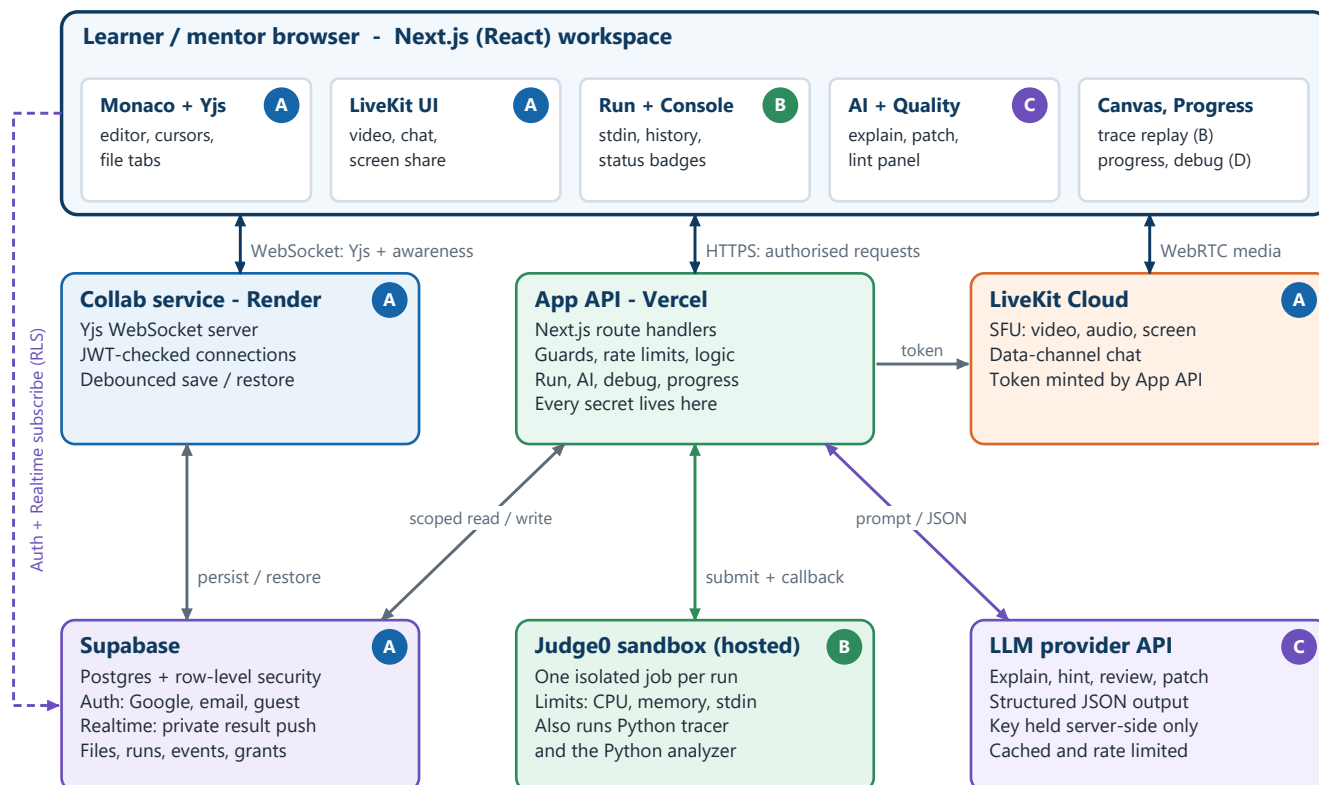
Group	Items	Why deferred
Classroom layer	Classroom system, rosters, invite codes, scheduled labs, announcements; student / teacher / admin dashboards; LMS integrations	Needs real institutional data and accounts. The mentor overview (T-A-09) and class trends (T-D-05) show the idea.
Collaboration extras	Breakout rooms, whiteboard, shared code-review threads, session recording and replay, waiting room and approvals	Each is a product in itself; none is required by the PS. Present as roadmap.
Execution extras	More than 5 languages, package installs, multi-file projects, interactive terminal, true step debugger with breakpoints	Judge0 runs batch jobs, so interactive process control is not available. The canvas replays observed states instead.
Grading and competition	Auto-grading, leaderboards, contest ranking, plagiarism detection, gamification	The compendium defines the product as a learning environment, not a judge. Deliberately out of scope.
Platform	Plugin system, public share links, localization, persistent threaded chat, notifications centre, admin console, audit log, full offline-first	Nice to have after the core loop is proven.

NOTE

Tier meanings: **P0** must ship, it is the problem statement. **P1** differentiator, build after P0 is working. **P2** polish or stretch. **P3** roadmap list only.

4. System architecture

4.1 Big picture



Coloured circles mark the lane that owns each component (A, B, C). Lane D owns the UI shell and the learning/debug features that sit on top.

Figure 1 - Components, hosts and the lane that owns each. Arrows show who talks to whom.

- **Code and media travel separately.** Edits go browser to Yjs service; video goes browser to LiveKit. Neither passes through the App API.
- **Private results use the database, not the WebSocket.** A run result is written to Postgres and pushed to its owner by Realtime, filtered by row-level security.
- **The App API is the only place that holds secrets** and the only thing that talks to Judge0, the LLM and LiveKit's server SDK.

4.2 Components

Component	Responsibility	Technology	Host	Lane
Web app	Every panel: editor, video, console, AI, quality, canvas, progress	Next.js App Router, React, TypeScript, Tailwind, Monaco	Vercel	A (shell D)
Collab service	Yjs sync and awareness, JWT check, persistence	Node; y-websocket or Hocuspocus (decision D6)	Render web service	A
App API	Guards, run, trace, AI, patch, debug grants, playback, checkpoints, progress	Next.js route handlers with zod validation	Vercel	all, per route
Database	Source of truth, RLS, Realtime push of private rows, Auth	Supabase Postgres, Auth, Realtime	Supabase	A (schema)
Communication	Video, audio, screen share, chat	LiveKit Cloud; livekit-server-sdk, livekit-client, components-react	LiveKit Cloud	A
Execution	Runs student code in isolation; also runs the tracer and Python analyzer	Judge0 REST (language ids read from /languages)	Judge0 provider or small VM	B

Component	Responsibility	Technology	Host	Lane
AI gateway	Explain, hint, review, patch, follow-up variants	Provider-agnostic adapter; fast and strong model	LLM API	C
Harness scripts	Python tracer and analyzer that run inside the sandbox	Python standard library only	Repo /harness	B and C

4.3 Trust boundaries

- The **browser is untrusted**. Every API route re-checks identity, room membership and role. Hiding a button is never the control.
- **Secrets** (service-role key, Judge0 key, LLM key, LiveKit secret, JWT secret) exist only in Vercel and Render environment variables.
- **Student code runs only inside Judge0**. Our own servers never execute it.
- **Private data** (runs, traces, explanations, checkpoint answers) is protected twice: API checks and Postgres RLS. Supabase Realtime honours RLS.
- The **collab WebSocket** needs a short-lived JWT minted after a membership check; viewers are read-only.
- **Prompt injection**: student code and error text go to the model as quoted data, never as instructions; model output is schema-validated before use.
- **AI changes never auto-apply**. They pass through a diff the student must accept.

HOSTING REALITY CHECK

Render web services can sleep when idle (slow first request); Supabase free projects can pause after a week idle; LiveKit, Judge0 and LLM providers have quotas. Vercel functions cannot hold WebSockets, which is why the collab service is separate. Check current limits in T-00-05; add keep-warm pings in T-A-12.

4.4 Technology choices

RECOMMENDED STACK - PLEASE CONFIRM

The compendium fixes the **requirements** (CRDT sync, containerized sandbox, role-aware permissions, optional Supabase Auth and Row Level Security) but not the products. Everything in this table is **my recommended stack** for four people and a short build. Decision D12 asks the team to confirm or swap it before phase 0.

Layer	Choice	Why
Editor	Monaco via @monaco-editor/react; y-monaco binding	VS Code feel, built-in diff editor for patch preview, mature Yjs binding
Sync	Yjs CRDT, y-protocols awareness	Conflict-free merges (required by the compendium); offline edits merge on reconnect
Backend	Next.js route handlers, zod, diff-match-patch	One deployable, typed contracts shared with the UI
Data	Supabase: SQL migrations in /supabase, RLS, Realtime, anonymous sign-in	Auth + DB + push in one; anonymous sign-in makes judge access trivial
Video	LiveKit Cloud	WebRTC SFU with ready React components, screen share and data channel chat
Execution	Judge0	One job per run, limits built in, five languages, no Docker work for us
Quality	ESLint Linter API, Python ast, compiler warnings	Real analysis for free; LLM only explains findings
Testing	Vitest for pure logic; Playwright or checklist for the golden path	Test parsers, status map and tracer output; do not over-invest in UI tests

4.5 Repository layout

syncverse/	
apps/web/	Next.js app: pages, panels, API route handlers
services/collab/	Yjs WebSocket service (deployed to Render)
packages/shared/	zod schemas, types, fixtures, constants (every lane imports this)
supabase/	migrations, RLS policies, seed scripts
harness/	python tracer + python analyzer (run inside the sandbox)
docs/	ARCHITECTURE.md, tracker/lane-a..d.md, STATUS.md
CLAUDE.md	conventions each Claude session reads first

5. Core flows

Four flows carry most of the product. Each diagram is the contract between lanes: if a step is unclear, fix it here before anyone codes it. Dashed arrows are responses or pushes.

5.1 Private code run (lane B)

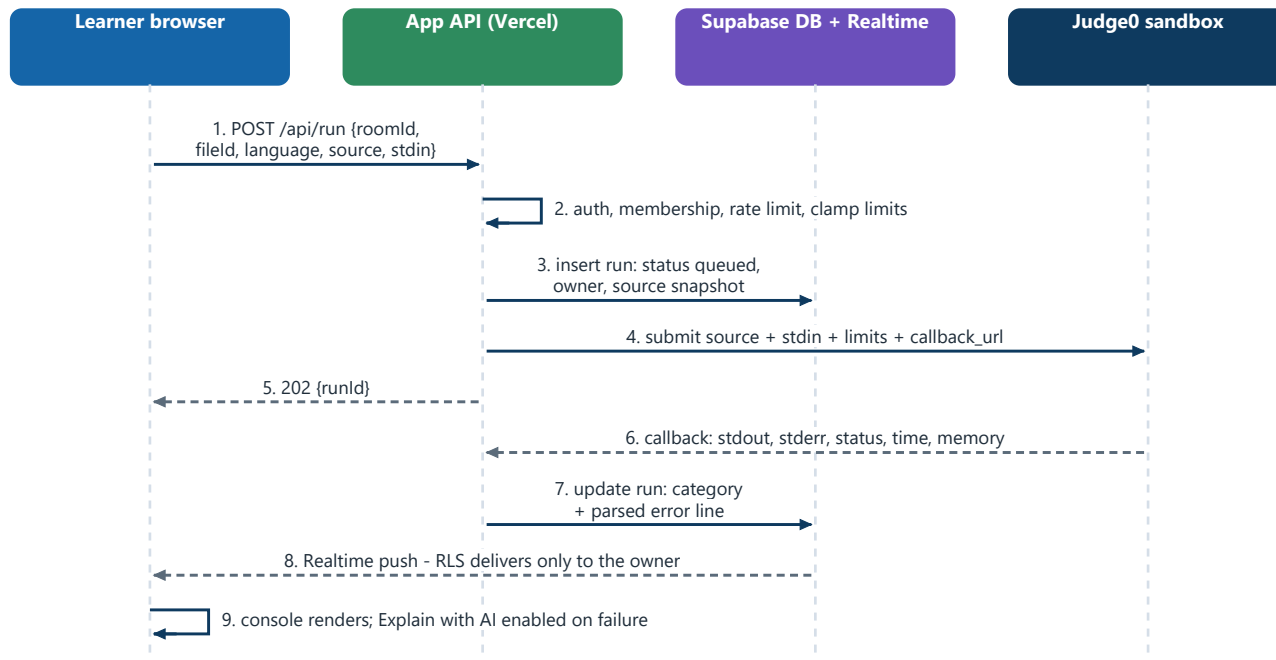


Figure 2 - If no callback arrives within about 3 s, `GET /api/run/:id` refreshes the result from Judge0 (lazy pull), so a lost callback never leaves a run stuck.

5.2 AI explanation and safe patch (lanes C, A)

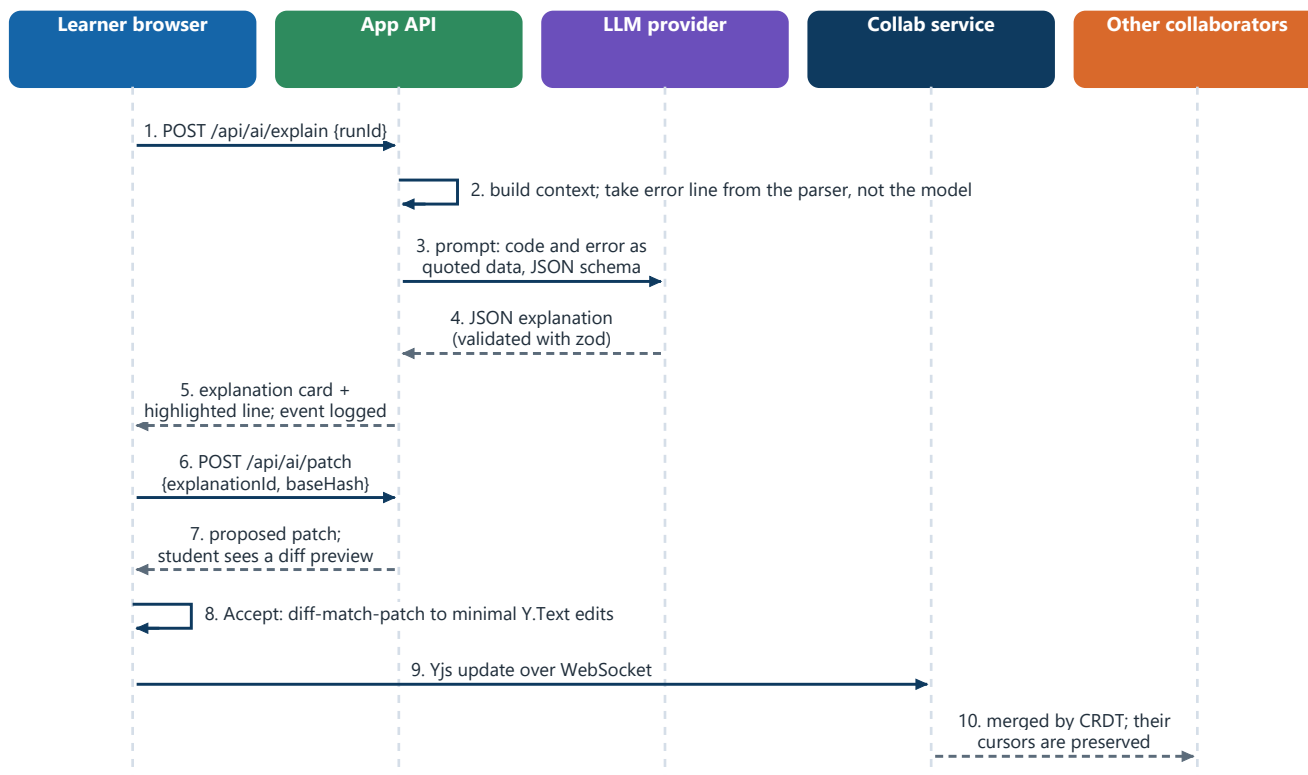


Figure 3 - Reject sends nothing to the collab service. If the document changed since `baseHash`, the preview is refreshed before it can be accepted.

5.3 Permission-gated debugging (lane D)

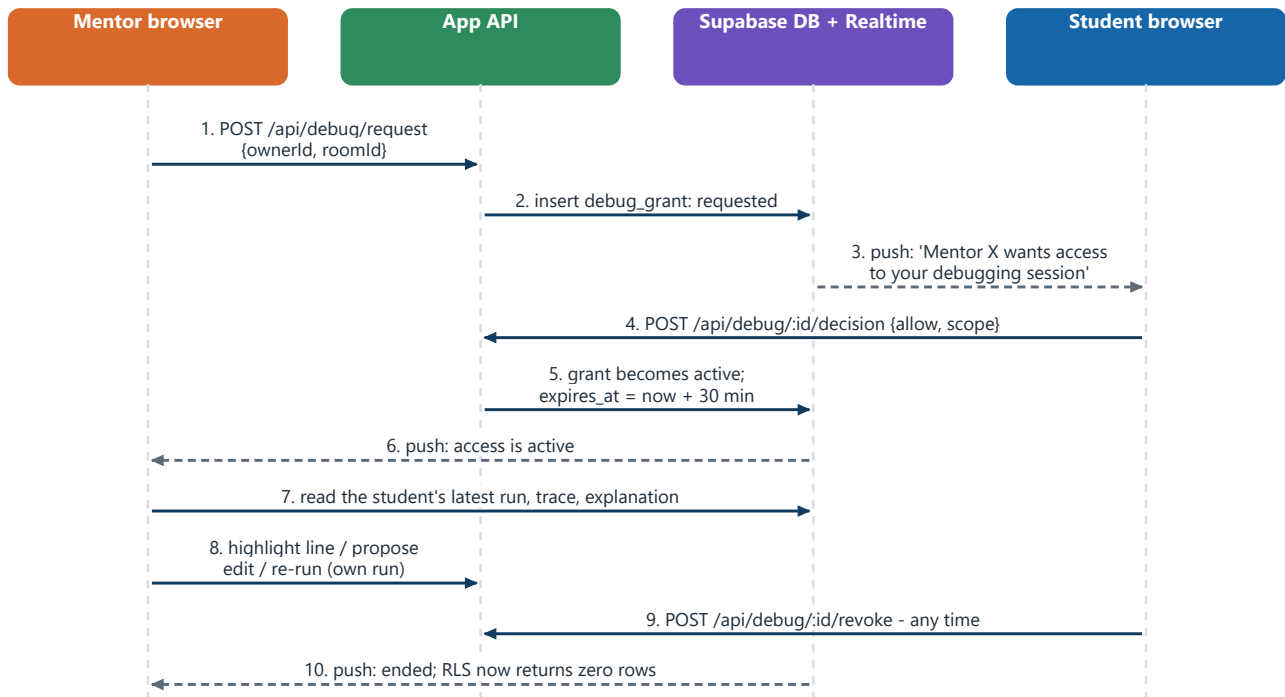


Figure 4 - Step 7 succeeds only while a matching active grant exists. Re-run creates a new run owned by the mentor, so the student's console is never changed.

5.4 Trace, shared playback, predict and reflect (lanes B, D)

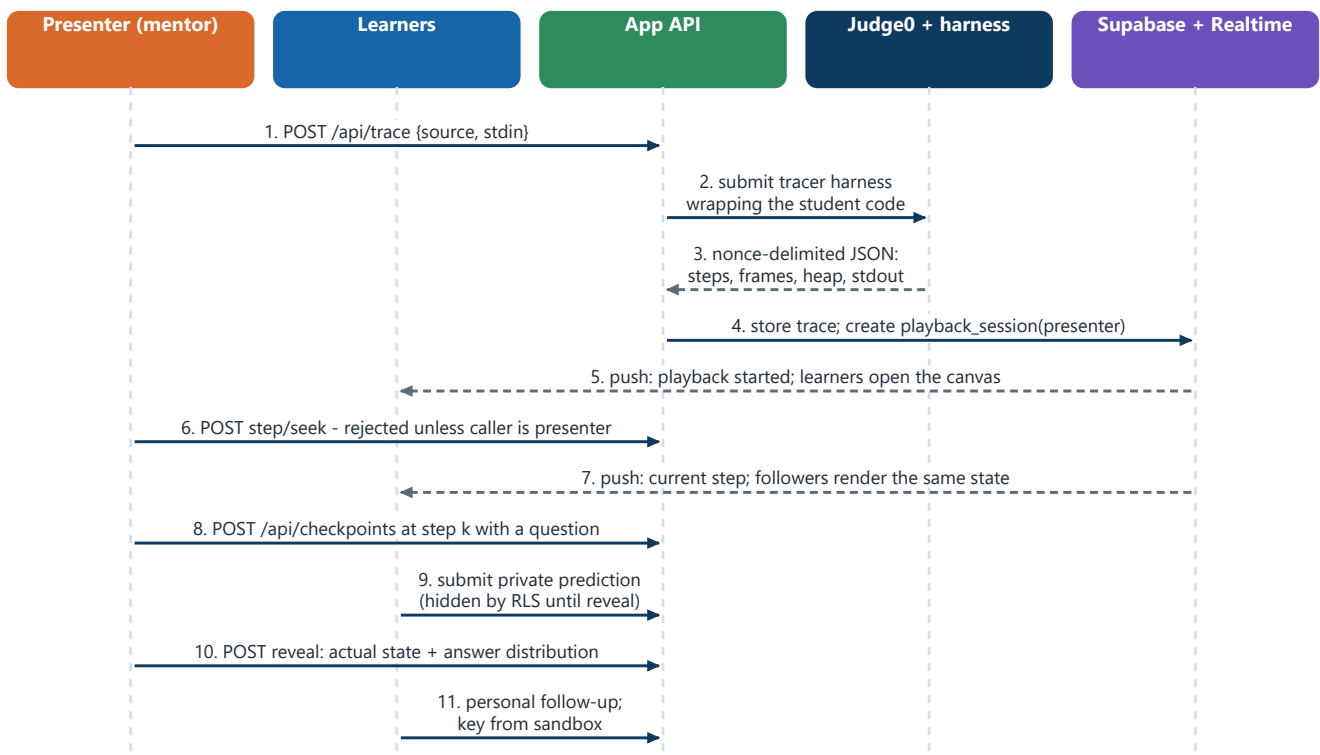


Figure 5 - The model may propose a follow-up variant, but the correct answer always comes from running it in the sandbox.

6. Feature deep dives

6.1 Real-time collaboration (lane A)

- **Document model.** One Y.Doc per file, named `file:{fileId}`, with a single Y.Text. File names and languages live in Postgres (source of truth) and reach other clients through Supabase Realtime.
- **Connecting.** The client calls `/api/collab/token`; the API checks membership and returns a JWT (subject, room, role, about 10 minutes). The collab service verifies it on connect. Viewers connect read-only.
- **Persistence.** Load the stored Yjs state on first connect; save debounced (about 2 s) and on last disconnect. Kill the service mid-edit and nothing is lost.
- **Running uses the editor text.** The browser sends its current source with the run request and the API stores that snapshot. The server never needs to read the Yjs document.
- **Presence.** Awareness carries name, colour, cursor, active file and a typing / idle flag (idle after 30 s without input).
- **Reconnect.** The provider reconnects automatically; show a banner after 3 s offline. Yjs buffers local edits and merges them on return.
- **Server choice (D6).** y-websocket (the reference Yjs server) or Hocuspocus (auth and persistence hooks built in). Both are 'a Yjs WebSocket service on Render'; decide after a 2-hour spike at the start of T-A-03.

6.2 Sandboxed execution (lane B)

- **Independence is the rule.** Each click on Run creates a separate Judge0 submission with its own stdin. Three learners can run the same file at the same instant without seeing each other.
- **Limits (our policy, clamped to the server maximum).** about 5 s CPU, 128 MB memory, stdout capped at 64 KB and marked 'truncated', stdin capped at 10 KB. Network access off.
- **Delivery.** Judge0 sends an HTTP PUT to the callback URL, which carries a token signed with our secret (Judge0 itself does not sign). The route verifies the token, updates the run row, and Realtime pushes it to the owner; a lazy refresh covers a lost callback.
- **Categories.** Judge0 statuses are normalized into Success, Compilation Error, Runtime Error, Timeout, Memory Limit Exceeded, Service Error (mapping in Appendix B). Each failure shows a badge and an Explain with AI button.
- **Error parsing.** Parsers for Python tracebacks, gcc, javac and Node turn raw text into `{category, line, col, message, raw}`. The AI gets this, and the editor marks the line, so we never depend on the model to find the line.
- **Java gotcha.** Judge0 saves Java as `Main.java`, so a beginner's `public class Foo` fails to compile. Rewrite the class name to `Main` on submit and tell the learner.
- **Rate limits.** About one run per 2 s and 20 per 10 minutes per learner, counted with a cheap database query (no extra Redis service).

6.3 Python tracer and execution canvas (lane B)

The tracer is a Python script we ship in `/harness`. It wraps the student's code, runs it under `sys.settrace` inside the Judge0 sandbox, and prints one JSON document. The canvas replays that document; nothing is executed in the browser.

```
{ "steps": [ {
  "i": 7, "event": "line", "line": 4, "depth": 1,
  "frames": [ { "func": "<module>", "line": 9, "locals": { "a": { "ref": 1 }, "b": { "ref": 1 } } },
    { "func": "average", "line": 4, "locals": { "total": 6, "i": 2 } } ],
  "heap": { "1": { "type": "list", "items": [1, 2, 3] } },
  "stdout": "start\n" } ],
  "truncated": false, "exit": "ok" }
```

- **Aliasing.** Containers are stored once in the heap by a stable small id; two names pointing at one list both hold `{ref:1}`, so the canvas draws two arrows to one object. This is the classic bug the demo shows.
- **Caps.** Stop recording after about 300 steps (program still finishes so stdout is complete), cut reprs at 80 characters, show at most 20 items per container, and show 'trace truncated at step N'.
- **Filtering.** Only frames from the student's file; hide builtins, modules and dunder names.

- **Noise control.** The final JSON is printed between markers containing a per-run random nonce, so student prints cannot break parsing.
- **Canvas panels.** Code with current line; frames and variables; heap objects with SVG arrows; stdout so far; controls (first, previous, next, last, play, scrubber, arrow keys). Changed values get an outline and a 'changed' tag, never colour alone. Calls and returns animate unless reduced motion is on. A table view gives the same data as text.
- **Shared playback.** A playback_sessions row holds trace, presenter and current step. Only the presenter may call the step endpoint (checked on the server). Followers subscribe through Realtime; a learner can detach to explore and press 'Rejoin presenter'. A mentor can take over when the presenter releases.
- **Why not a real debugger?** Judge0 runs batch jobs; there is no interactive process to pause. Replay of observed states gives the teaching value. Breakpoint-style 'jump to next time line N runs' is a cheap later addition.

6.4 AI tutor (lane C)

Mode	Trigger	Input sent	Output	Tier
Explain error	Explain with AI on a failed run	file, language, NormalizedError, stdin, learner level	structured explanation, line, concepts	P0
Patch	Show patch on an explanation	same + explanation	corrected code shown as a diff	P0
Hint ladder	Hint button / room hint mode	same	tier 1 nudge, tier 2 question, tier 3 near-answer	P1
Explain code	select code, Explain	selection + file	purpose, flow, line-by-line	P1
Review	Review button	file	logic and structure notes with reasons	P1
Follow-up variant	mentor reveals a checkpoint	trace step, concept	variant program; answer key comes from the sandbox	P1
Explain step	Ghost Driver on the canvas	selected runtime state	short spoken-style explanation	P2
Tests and complexity; micro-lesson	buttons	file / weak concept	edge-case inputs, complexity note, lesson	P2

Structured explanation returned by the gateway and validated before display:

```
{ "what": "Your program stopped with an IndexError.",
  "where": { "line": 4, "source": "parser" },
  "why": "range(len(nums)+1) makes i reach len(nums), one past the last index.",
  "plain": "A list with 3 items has positions 0, 1, 2. Position 3 does not exist.",
  "fix": "Stop the loop one step earlier: range(len(nums)).",
  "snippet": "for i in range(len(nums)):",
  "concepts": ["loop-boundaries", "lists-arrays"], "confidence": "high" }
```

- **Context policy.** Send only the active file (trimmed to about 80 lines around the error when long), language, normalized error, stdin and learner level. Never other users' code, names or emails.
- **Guardrails.** Code and errors are delimited data; output must pass the zod schema (one retry, then a plain fallback message); the UI always says the AI can be wrong; per-user rate limit and daily token budget; hint tiers 1 and 2 must not contain corrected code.
- **Cost and speed.** Default to a fast, cheap model for explain and hint; use a stronger model for patch and review; stream text; cache by hash of language + normalized error + code; pre-bake answers for the demo programs.
- **Patch apply.** Compute a minimal diff against the student's current text, apply it as a Yjs transaction so other cursors are undisturbed, and block accept if the document changed since the patch was generated.
- **Evaluation.** A 20-case set (T-C-10) with a rubric: right cause, right line, beginner wording, no full answer in hint tiers 1-2, no invented APIs. Re-run it whenever a prompt changes.
- **Model choice (D3).** Provider-agnostic adapter configured by environment. If the team uses Claude, the fast tier would be Haiku 4.5 (c1aude-haiku-4-5-20251001) and the strong tier Sonnet 5.5 (c1aude-sonnet-5-5); any provider with JSON output works.

6.5 Code quality analysis (lane C)

Real analyzers find problems; the model only explains them. Every analyzer implements `analyze(source, language) -> Diagnostic[]` with `{category, severity, line, col, endLine, endCol, rule, message}`.

Category	Example rules	How (by language)
Formatting	line over 100 characters, mixed tabs and spaces, trailing whitespace, inconsistent indent	generic text rules for all languages
Naming	one-letter names outside loops, style mismatch (snake_case, camelCase), very long names	JS: ESLint + rules; Python: ast; C/C++/Java: regex
Code smells	duplicated blocks, nesting deeper than 3, function over 30 lines, dead code after return, unused variable or import, magic numbers, empty except/catch	ast for Python; ESLint for JS; heuristics (brace depth, hashing) for C-family
Complexity	nested loop depth, cyclomatic complexity per function, plain-language time note	same as above
Security hints (educational)	eval / exec, hard-coded secrets, SQL built by concatenation, gets / strcpy, os.system, pickle.loads	pattern rules per language
Compiler warnings	-Wall -Wextra output for C and C++, javac lint notes	Judge0 compile_output, parsed

- **When it runs.** Cheap generic rules about 1.5 s after typing stops; the full analysis on Run and on an explicit Analyze button.
- **Where it shows.** Monaco markers, a Problems list grouped by category, and a maintainability summary with five dimensions (readability, structure, naming, duplication, potential bugs), each 'Looks good', 'Some notes' or 'Needs attention' with counts.
- **No single score.** The panel says 'feedback for learning - not a grade', matching the compendium.
- **Python analyzer** is a stdlib-ast script in /harness run through the same Judge0 pipeline, so the API needs no Python runtime.

6.6 Permission-gated debugging (lane D)

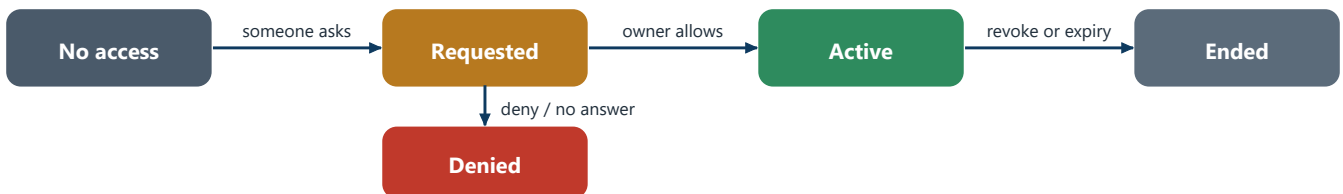


Figure 6 - States of a debug grant. One active grant per (owner, grantee) pair.

- **Private by default.** Execution and debugging belong to the learner. A teacher role alone grants nothing.
- **Scope.** The student chooses 'View only' or 'View and assist'. Assist adds highlight line, propose edit (arrives as a patch the student must accept) and re-run (creates a new run owned by the helper).
- **Enforcement.** The API validates every call and an RLS policy lets a grantee read the owner's runs, traces and explanations only while an active, unexpired grant exists. Revoke takes effect on the next read.
- **Safety.** Grants auto-expire after 30 minutes or when the student leaves; at most 3 requests per requester per 10 minutes; 'Block requests from this person' for the session; a visible banner on both sides while access is active.
- **Peers too.** Anyone in the room can request access to anyone; the owner decides.
- **What a mentor sees without a grant.** Presence, active or idle, last run category, failed-run streak, time since last success. The UI tells students this ('your mentor can see that you are stuck, not what you ran'). This is decision D5.

6.7 Predict and reflect (lane D)

1. A presenter places a checkpoint on a trace step and writes or accepts a suggested question ('What will total be after this line?'). Answer type: number, short text or multiple choice.
2. Each learner answers privately. Answers are stored under RLS so only the author can read them until the presenter reveals.
3. The presenter sees a live 'answered 3 of 4' count. Nobody sees anyone's answer yet.
4. Reveal: the canvas advances to the real state; learners see their answer beside it; the mentor sees the per-person spread; learners see the class distribution without names.
5. Each learner gets an individual follow-up: the model proposes a variant of the same idea (different numbers or a small twist); the sandbox runs it to compute the correct answer key; the learner answers alone.
6. Answers and follow-up results are logged as learning events tagged with the concept. Nothing is scored or ranked; the progress view turns them into observations.

This is the mechanism that turns 'the pair got the right answer' into 'each person showed they understood why', which is the compendium's second core problem: shared code hiding unequal understanding.

6.8 Progress tracking and analytics (lane D)

Event	Emitted by	Key payload	Used for
run_finished	B	category, language, duration	success rate, error categories
ai_explain_requested	C	error category, concepts	help topics, recurring errors
hint_used	C	tier	reliance on hints
patch_decision	C	accepted or rejected	engagement with AI help
lint_fixed	C	category, count	quality trend
checkpoint_answered / followup_answered	D	concept, correct	individual concept evidence
debug_access	D	requested, allowed, denied	how often help was needed
heartbeat	A	active seconds per minute	coarse time in room

- **Observations, not scores.** A concept appears under 'Retry recommended' when it has at least 3 evidence items in 7 days and fewer than two thirds were successful. Thresholds live in packages/shared so they can be tuned.
- **Student view.** Run and success trend, error categories over time, concepts practised, observation sentences, suggested next concepts, and a 'what we record' panel.
- **Mentor view.** Concept by student matrix (numbers plus patterns, not colour alone), common errors, most-asked AI topics, active and inactive learners, average debugging time, click-through to one learner's progress.
- **Privacy.** No keystroke logging, no code diffs, no cursor trails. Events are coarse. Mentors see aggregates and status; content needs a grant.
- **Honest claims.** The compendium frames progress as evidence of engagement, not a certified measure of mastery. Keep that wording in the demo; if you want a number, run the optional mini-pilot in T-X-05.

7. Data model, permissions and API

7.1 Tables (Supabase Postgres)

Table	Purpose and key columns	Read (RLS)	Write
profiles	id (auth user), display_name, color, role student mentor admin	members of a shared room	self
rooms	id, code (unique), name, owner_id, settings (hint mode, level)	members	API
room_members	room_id, user_id, workspace_role owner editor viewer	members	API
files	id, room_id, name, language, content_mirror, updated_at	members	API, collab
yjs_docs	file_id, state (bytea), updated_at	none (service only)	collab
snapshots	id, file_id, content, label, created_by, created_at	members	API
runs	id, room_id, file_id, user_id, source, language, stdin, status, category, stdout, stderr, compile_output, time_ms, memory_kb, parsed_error jsonb	owner or active grantee	API
explanations	id, run_id, user_id, kind, request jsonb, response jsonb, concepts[]	owner or active grantee	API
patches	id, explanation_id, file_id, base_hash, diff, status proposed accepted rejected, decided_by	owner or active grantee	API
quality_reports	id, file_id, user_id, diagnostics jsonb, summary jsonb, created_at	owner or active grantee	API
debug_grants	id, owner_id, grantee_id, room_id, scope, status, requested_at, decided_at, expires_at, revoked_at	owner and grantee	API
traces	id, run_id, steps jsonb, step_count, truncated	owner, grantee, or room when presented	API
playback_sessions	id, room_id, trace_id, presenter_id, current_step, updated_at	members	API only
checkpoints	id, trace_id, step_index, question jsonb, concept, revealed_at	members	presenter via API
checkpoint_answers	id, checkpoint_id, user_id, answer jsonb, correct, followup jsonb	author only until revealed , then mentor	member via API
concepts	slug, name (seeded; Appendix C)	all signed-in	seed
learning_events	id, user_id, room_id, type, concept_slugs[], payload jsonb, created_at	self; mentor aggregates via views	API
help_requests (P2)	id, user_id, room_id, reason, created_at	mentors in room	API

7.2 Who can do what

Action	Mentor	Student (editor)	Observer (viewer)
Create room, invite	Yes	Yes	No
Edit shared code	Yes	Yes	No (read-only)
Run code (private run)	Yes	Yes	Yes
See another person's runs, traces, explanations	With grant	With grant	With grant
See others' activity status (run state, stuck flag)	Yes	No (presence only)	No
Request debug access to someone	Yes	Yes	Yes
Allow, deny or revoke access to own session	Own session	Own session	Own session
Ask the AI (explain, hint, review)	Yes	Yes	Yes
Accept an AI patch into the shared file	Yes	Yes	No (may preview)
Present and control shared playback	Yes	After handoff	No

Action	Mentor	Student (editor)	Observer (viewer)
Create predict checkpoints	Yes (as presenter)	No	No
See class trends	Yes	No	No
See own progress	Yes	Yes	Yes

Identity role (student or mentor, on the profile) and workspace role (owner, editor, viewer, on the room membership) are separate on purpose, so being a teacher never grants silent control.

7.3 API routes

Route	Purpose	Guard	Lane
POST /api/rooms, /api/rooms/join	Create room; join by code	signed in	A
GET /api/collab/token	Mint Yjs JWT	member	A
GET /api/livekit/token	Mint LiveKit token	member	A
POST /api/run, GET /api/run/:id	Create and poll a run	member; rate limited; owner or grantee	B
PUT /api/judge0/callback	Receive Judge0 result	signed token in the URL	B
POST /api/trace	Run tracer harness	member; rate limited	B
POST /api/playback, /:id/step, /:id/release	Claim presenter; step; release	mentor or editor; presenter only for step	B
POST /api/ai/explain hint explain-code review patch	AI modes	owner or grantee; rate limited	C
POST /api/patches/:id/decision	Accept or reject a patch	requester (editor)	C
POST /api/analyze	Run analyzers	member	C
POST /api/debug/request, /:id/decision, /:id/revoke	Debug access lifecycle	member; owner for decision	D
POST /api/checkpoints, /:id/answer, /:id/reveal; POST /api/ai/followup	Predict and reflect	presenter; member; presenter	D
GET /api/progress/me, /api/progress/room/:id	Progress views	self; mentor	D
GET /api/health	Smoke test of every integration	public	A

7.4 Realtime channels

- runs filtered to user_id = me: private result push (plus rows of owners who granted me access).
- debug_grants where I am owner or grantee: request prompts and access changes.
- playback_sessions and checkpoints for the room: presenter step and reveal events.
- files for the room: file list changes. Document text itself uses the Yjs connection.

8. UX blueprint

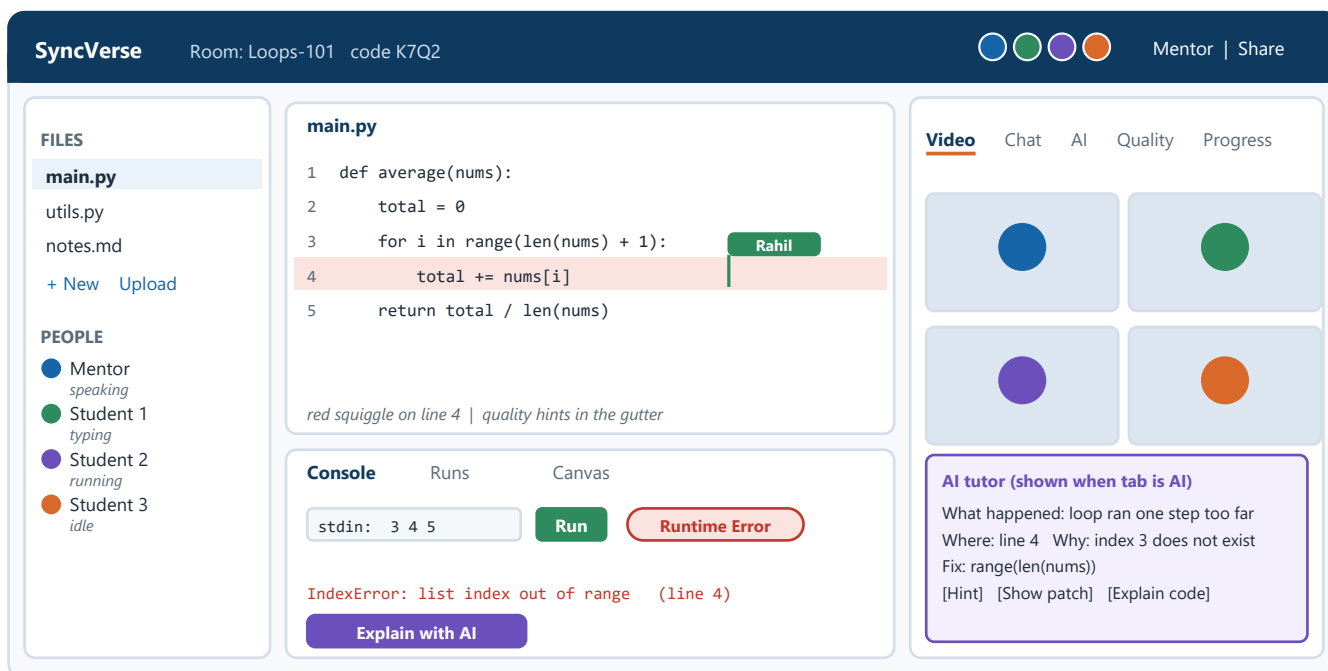


Figure 7 - Workspace layout. Editor centre, files and people left, dock right (Video, Chat, AI, Quality, Progress), bottom panel (Console, Runs, Canvas).

- **Nothing forces a tab switch.** The video dock collapses to a strip so the editor can use the full width; the AI card can be pinned while the learner types.
- **Canvas mode.** The bottom panel can expand to a full-height split: code on the left, canvas on the right. Presenter controls appear in the top bar.
- **Private vs shared is always visible.** Console and Runs show a lock with 'only you'; a banner appears whenever someone has debug access.
- **Beginner tone.** Labels and explanations avoid jargon; concept names link to a one-line definition.
- **Desktop first** (1280 px and up); usable on a tablet; phones are out of scope.

Screen	Purpose	Lane
Landing	Pitch, join with code, sign in or continue as guest	D
Sign in and first-run	Auth, pick role, name and colour, three-step tour	A, D
Rooms	Create or join; recent rooms	A
Workspace	The layout above	all
Canvas (expanded)	Trace replay, playback, checkpoints	B, D
My progress	Observations, concepts, trend	D
Mentor overview and class trends	Status grid, request access, concept matrix	A, D
Settings	Hint mode, learner level, what we record	C, D

Accessibility and reach (baseline, from the compendium)

- Full keyboard operation including canvas stepping; visible focus; labels on every control.
- No information by colour alone (status badges carry text, changed values carry a tag).
- Reduced-motion setting; captions and mute controls wherever audio is used.
- Low-bandwidth mode (P2): audio-only, code sync always first.

9. How the team works

9.1 Your question: divide the work, or keep a sheet and take turns?

RECOMMENDATION

Do both, but not as a relay. Give each person one **lane** that they own end to end, and keep a **tracker in git** that every lane updates. Run short integration windows at the end of each phase. Use 2-hour turns only as a fallback if you must share one machine or one AI account.

	Relay: one person at a time, 2 h each	Parallel lanes (owned)	Hybrid - recommended
Throughput	One person's worth at any moment	Four people at once	Four people at once
Context	Lost at each handoff unless notes are perfect	Stays with the lane owner	Stays with the owner; the tracker carries cross-lane context
Merge conflicts	None, but branches go stale	Frequent if people touch the same files	Rare: lanes own directories, contracts live in packages/shared
Can everyone explain the whole system in Q and A?	Everyone knows a little, nobody owns a part	Each person knows one lane only	Owner plus a named backup per lane; integration windows spread knowledge
Needs	One machine or one account	A machine each	A machine each, plus a git remote

Why the sheet must live in git. A Claude session only knows what is in its prompt and in your repository. It does not see what a teammate's session did. If progress and decisions are not written into `CLAUDE.md`, the tracker files or `docs/`, the next session starts blind. Git is also the only place all four laptops can share. The folder `E:\SyncVerse` is empty and not a repository yet; T-00-02 fixes that.

9.2 Lanes

Lane	Owns	Suits someone who likes	P0	P1	P2	Total	Owner	Backup
A	auth, rooms, editor, Yjs server, files, presence, video and screen share, mentor grid, ops	infrastructure, realtime systems, deployment	14.5	2.5	2.0	19.0	_____	_____
B	Judge0 pipeline, run UI, parsers, tracer, execution canvas, shared playback	backend plumbing and visual front-end (SVG), algorithms	8.5	9.5	2.0	20.0	_____	_____
C	LLM gateway, explainer, patches, hints, review, lint engine, quality panel, AI eval	prompts, static analysis, correctness of text output	11.0	5.0	2.5	18.5	_____	_____
D	design system, debug permissions, events, progress views, predict-and-reflect, demo data, chat, landing	product thinking, UI polish, data views, privacy details	10.5	9.0	1.5	21.0	_____	_____
X	whole-team tasks: accounts, smoke test, freeze, rehearsals, submission	everyone	6.0	0.0	0.0	6.0	all	-

Blocks are person-blocks of about 2 hours. One person (your choice) acts as integration lead and owns cross-lane decisions and the integration windows, whatever their lane. Pick lanes by preference; the totals are balanced to within 2.5 blocks.

9.3 Rules everyone follows

1. **Contracts first.** packages/shared changes only by a PR all four lanes have seen. Prefer additive changes.
2. **Trunk-based git.** One branch per task named lane-b/T-B-03-private-runs; PR under a day old; squash-merge to main. Main deploys a preview; nobody leaves main broken.
3. **Mock first.** Build against fixtures until the providing lane ships. Never wait idle on another lane.
4. **Feature flags.** Unfinished P1 and P2 panels hide behind NEXT_PUBLIC_FF_* so the demo build is always clean.
5. **Ten-minute review.** A teammate from an adjacent lane skims each PR within 2 hours; purpose is to catch contract breaks, not style.
6. **Integration window.** At the end of every phase, 90 minutes where everyone merges, runs the golden path and logs bugs against lanes.
7. **Daily sync.** Ten minutes: done, doing, blocked. Anything blocked is written in the tracker with 'needs: lane X'.
8. **Own your directory.** Edits outside your lane are announced in the tracker first.
9. **Definition of done** is Appendix D; a task is not DONE until it passes on the deployed preview.

9.4 The tracker

```
docs/
  ARCHITECTURE.md      condensed copy of this blueprint
  tracker/
    lane-a.md lane-b.md lane-c.md lane-d.md shared.md    one file per lane (no merge fights)
    STATUS.md          roll-up: counts per lane and phase, open blockers (regenerated by a script)
  CLAUDE.md            stack, conventions, 'read the tracker first', ownership map
```

```
### T-B-03 Private results and realtime delivery      [TODO | DOING | REVIEW | DONE | BLOCKED]
Owner: _____ Started: _____ Branch: lane-b/T-B-03-private-runs
Needs: T-B-01, T-00-04                               Blocked by: -
Done when: no browser ever receives another learner's result
Handoff: what is finished, what is next, gotchas
```

How a teammate's Claude session uses it (examples of what to type):

- 'Read CLAUDE.md and docs/tracker/lane-b.md. Take the next TODO task in my lane whose dependencies are DONE, set it DOING, implement it, then update the tracker with a handoff note.'
- 'T-B-03 is merged. Mark it DONE, regenerate docs/tracker/STATUS.md and list what it unblocks.'

9.5 Who depends on whom

Consumer	Needs from	What
B Execution	A	requireMember guard, rooms, EditorHandle (markers, highlightLine)
C AI and Quality	B	NormalizedError from the parsers, run rows, planted-bug sample programs
C AI and Quality	A	EditorHandle.applyEdits and Y.Text access for the patch flow
D Learning	B	runs, traces and playback for the debug mirror and checkpoints
D Learning	C	patch flow for mentor-proposed edits; concept tags
A Platform	B, D	mentor grid reads run status, debug grants and events
B Execution	C	Ghost Driver calls the AI gateway

9.6 Fallback: only one machine or one account

- Keep the same lanes and tracker. A 'turn' is one task card (at most 2 hours); whoever is on turn takes the next TODO whose dependencies are DONE, writes a handoff note before the turn ends, and never leaves a task half-merged on main.
- For round-the-clock coverage, hand a lane's tracker file to the next person on shift; do not reassign lane ownership mid-phase.

10. Phased plan and backlog

10.1 Timeline by phase and lane

Each cell lists task IDs (without the T- prefix) and the person-blocks in that phase. Within a phase, tasks run in parallel against fixtures; order matters only between phases.

Phase	Goal	A	B	C	D	Team
0 Kickoff and contracts	Accounts, repo, shared types, DB schema, deploy a smoke test to every real host. Nobody writes feature code before contracts exist.	00-02 00-04 2.0	00-03 1.0	-	D-01 2.0	00-01 00-05 1.5
1 Walking skeleton	Login -> create/join room -> two browsers edit one file -> each runs Python and sees only their own output.	A-01 A-02 A-03 A-04 7.0	B-01 B-02 4.0	C-01 C-04 4.5	D-03 1.5	-
2 PS-complete (all P0)	All seven bullets of the problem statement work at a basic level: sync, video, AI explain + patch, sandboxed run, quality, permissioned debug, progress.	A-05 A-06 A-07 4.5	B-03 B-04 B-05 B-06 6.5	C-02 C-03 C-05 6.5	D-02 D-04 D-12 6.0	-
3 Differentiators (P1)	Execution canvas, shared playback, predict-and-reflect, teacher overview, class trends, hint mode, code review.	A-09 2.5	B-07 B-08 6.5	C-06 C-07 C-08 C-09 C-10 5.0	D-05 D-06 6.0	-
4 Polish and hardening (P2 + a11y)	Accessibility, reconnect/keep-warm, version history, low-bandwidth, demo seed data, landing/onboarding, stretch items.	A-10 A-11 A-12 3.0	B-09 B-10 2.0	C-11 C-12 2.5	D-07 D-08 D-09 D-10 D-11 5.5	-
5 Freeze and demo	Feature freeze, golden-path tests, rehearsals, backup video, submission pack.	-	-	-	-	X-01 X-02 X-03 X-04 X-05 4.5
Total		19.0	20.0	18.5	21.0	6.0

10.2 Milestones

	When	Exit criteria
M0	End of Phase 0	Every host answers on /api/health; all four leads have signed off on packages/shared.
M1	End of Phase 1	Two browsers edit the same Python file live; each clicks Run with different stdin and sees independent output.
M2	End of Phase 2	Every PS bullet is demonstrable (even if rough): editor, video, AI explain + patch, run, lint, debug-grant, progress.
M3	End of Phase 3	The signature flow works end to end: trace -> shared playback -> predict -> reveal -> progress evidence.
M4	Feature freeze	No new features. Only bug fixes, copy, performance, demo data. Main is always deployable from here.
M5	Demo-ready	Three clean rehearsals on the real URLs; backup video recorded; submission pack complete.

10.3 If time runs short: what to cut, in order

1. **All P2** (8.0 blocks): Ghost Driver, canvas accessibility extras, low-bandwidth mode, version history, test-case suggestions, practice problems, stuck nudge, session summary.
2. **T-D-08** landing page (keep join-with-code and guest mode) and trim **T-D-11** to keyboard and contrast only.
3. **T-C-07** code explainer and **T-C-08** AI review.
4. **T-D-05** class trends (keep the student progress view and the mentor grid).
5. **T-B-08** shared playback (keep the single-user canvas; the mentor can screen-share it).
6. **T-D-06** predict and reflect, only if the canvas is also dropped.

RECOMMENDATION

Protect: every P0 task, plus the trace and canvas (T-B-06, T-B-07). The canvas is the most visible differentiator and the compendium lists it first among the signature ideas (section 3.2).

10.4 Backlog by lane

Tier: P0 must ship, P1 differentiator, P2 polish. Ph is the phase. Est is person-blocks of about 2 hours. Every task ends with a 'Done when' line that is its acceptance test.

Shared - whole team (6.0 blocks, 7 tasks)					
ID	Task and deliverable	Tier	Ph	Est	Needs
00-01	Accounts, keys and .env.example Create Vercel, Render, Supabase and LiveKit Cloud projects; get Judge0 access (hosted key or a small VM) and an LLM API key. Secrets live in a shared password manager, never in git. Done when: every lane can read its keys and <code>.env.example</code> lists every variable (Appendix A).	P0	0	0.5	-
00-05	Integration smoke test on real hosts Deploy a <code>/api/health</code> page proving: Supabase read, Yjs WebSocket echo on Render, LiveKit token, 4 concurrent Judge0 runs, one LLM call. Each lane verifies its own service. Done when: all green on deployed URLs, not localhost.	P0	0	1.0	00-01 00-02
X-01	Golden-path test script A written checklist (and Playwright for the three riskiest steps if time) that walks section 12.1 on the deployed URLs. Done when: two people can run it cold and report pass/fail.	P0	5	1.0	D-07
X-02	Feature freeze and production deploy Freeze scope, tag the release, prod env vars, warm-up checklist (ping Render, Supabase, Judge0, LiveKit, LLM 10 minutes before). Done when: the tagged build is the one everyone rehearses on.	P0	5	0.5	X-01
X-03	Demo script and three rehearsals Timed script, roles per person, three full run-throughs including one with a deliberate failure (Wi-Fi drop or slow AI). Done when: demo fits the slot with 30 s spare.	P0	5	1.5	X-02
X-04	Backup video and screenshots Screen-record the golden path with audio; export screenshots of every major screen. Done when: video plays from a USB stick offline.	P0	5	0.5	X-02
X-05	Submission pack README with architecture image and setup, pitch material updated to match what was built, live URL and demo credentials; optional 15-minute mini-pilot with 4-6 classmates (time to fix with and without AI explanation) so claims rest on evidence. Done when: a stranger can open the README and reproduce the demo.	P0	5	1.0	X-02

Lane A - Platform, Collaboration and Comms (19.0 blocks, 13 tasks)					
ID	Task and deliverable	Tier	Ph	Est	Needs
00-02	Repo and monorepo skeleton git init + remote; pnpm workspace with apps/web, services/collab, packages/shared, supabase/, harness/, docs/. Strict TypeScript with current stable package versions pinned in the lockfile, lint + typecheck CI, PR template, CLAUDE.md with conventions, one tracker file per lane. Done when: a fresh clone builds and CI is green.	P0	0	1.0	-
00-04	Database schema v1 and RLS baseline Migrations for every table in section 7.1, default-deny RLS, member-read policies, seed stub, generated TS types. Done when: two test users can each read only their own runs through the Supabase client.	P0	0	1.0	00-03
A-01	Auth, profiles and API guards Supabase Auth (Google, email link, anonymous guest for judges), profile with display name, colour and role (student / mentor), route guards; requireUser / requireMember helpers and a DB-count rate-limit helper used by every API route. Done when: a guest can enter a room in under 10 s.	P0	1	1.5	00-04
A-02	Rooms and membership Create room (name, short code), join by code or link, member list, workspace roles owner / editor / viewer, room list page. Done when: a second browser joins via code and appears in members.	P0	1	1.5	A-01
A-03	Yjs collaboration server on Render Start with a 2-hour spike: y-websocket vs Hocuspocus (decision D6). JWT-checked connections via /api/collab/token, one doc per file, read-only for viewers, debounced persistence to Supabase, restore on first connect, health endpoint. Done when: killing and restarting the service loses no text.	P0	1	2.0	A-01 00-04
A-04	Editor core (Monaco + Yjs) Monaco bound to Y.Text, language modes, line numbers, auto-indent, theme. Publishes an EditorHandle API other lanes depend on: getValue, getSelection, setMarkers, highlightLine, applyEdits, onSelectionChange. Done when: two browsers type at once with no lost characters.	P0	1	2.0	A-03 D-01
A-05	Presence and live cursors Labelled coloured cursors and selections, presence list (online / typing / idle, active file), join/leave toasts. Done when: a cursor label shows the right name within 300 ms.	P0	2	1.0	A-04
A-06	File management Tabs, new file, upload, rename, delete, extension to language detection, realtime file list. Done when: uploading a .py and a .java file gives correct modes on both browsers.	P0	2	1.5	A-04
A-07	Video, audio and screen share LiveKit token route and embedded panel: tiles, camera/mic toggles, screen share, collapse or float. Mentor can mute participants. Done when: 3 participants talk while editing without leaving the tab.	P0	2	2.0	A-02 D-01
A-09	Mentor overview grid One tile per participant: presence, last-run status, failed-run streak, idle time, stuck flag, Request-debug-access button, broadcast banner. Shows status only , never code output. Done when: a mentor spots a stuck student without opening their session.	P1	3	2.5	B-03 D-02 D-03
A-10	Snapshots and version history Manual and auto checkpoints (on every run), history list, Monaco diff view, restore. Done when: a student restores yesterday's version and collaborators see it.	P2	4	1.5	A-04
A-11	Low-bandwidth mode Audio-only toggle, adaptive streaming, code sync always prioritised. Done when: throttled to 3G the editor stays live while video degrades.	P2	4	0.5	A-07
A-12	Resilience and ops Reconnect banner, offline edit buffering check, keep-warm pings for Render and Supabase, env validation at boot, central error logging. Done when: a 60 s network drop recovers with no data loss and a clear banner.	P0	4	1.0	A-03

Lane B - Execution and Visualization (20.0 blocks, 11 tasks)

ID	Task and deliverable	Tier	Ph	Est	Needs
00-03	Shared contracts and fixtures zod schemas, TS types and JSON fixtures for Run, NormalizedError, Diagnostic, Explanation, Patch, DebugGrant, TraceStep, Checkpoint, LearningEvent; language config; error-category enum; concept taxonomy. Done when: all four lanes reviewed it and any UI can render from fixtures alone.	P0	0	1.0	-
B-01	Judge0 adapter and run pipeline Client wrapper (language map from /languages, base64, limits, compiler flags), POST /api/run storing a source snapshot and run row, HMAC callback plus lazy refresh on GET /api/run/:id, status to category mapping, Java class-name fix, per-user run rate limit. Done when: Python, C and Java each return correct stdout/stderr.	P0	1	2.5	00-03 A-01
B-02	Run UI and console Run button and shortcut, stdin box, console with separate stdout / stderr / compile tabs, status badge with time and memory, per-user run history with re-run. Done when: badge reads Compilation Error / Runtime Error / Timeout / Success correctly.	P0	1	1.5	B-01 D-01
B-03	Private results and realtime delivery Runs RLS owner-only, Realtime subscription filtered to own runs, test with 3 browsers running the same file with different stdin. Done when: no browser ever receives another learner's result.	P0	2	1.0	B-01 00-04
B-04	Error parsers and line highlighting Per-language parsers (Python traceback, gcc, javac, Node) produce NormalizedError {category, line, col, message, raw}; red squiggle and gutter marker in Monaco; click error to jump. Done when: the right line is marked for 10 sample errors per language.	P0	2	1.5	B-01 A-04
B-05	Language pack and sample programs Verify all five languages end to end, starter templates, stdin behaviour, limit messages (timeout, memory, output truncated), library of planted-bug sample programs reused by the demo. Done when: every language runs a hello-world and a stdin program.	P0	2	1.0	B-01
B-06	Python tracer harness sys.settrace harness records line/call/return/exception with locals, globals, stable heap ids (keeps aliasing), repr and step caps, per-step stdout, nonce-delimited JSON; POST /api/trace; test corpus (loop, recursion, list alias, mutation, exception, stdin). Done when: the aliasing demo program traces correctly and truncates cleanly at the cap.	P1	2	3.0	B-01
B-07	Execution canvas UI Code pane with current line, step controls + scrubber + keyboard, frames column, heap column with SVG arrows, changed-value highlight, call/return transitions, stdout-so-far, truncated-trace notice. Done when: a learner can explain list aliasing from the canvas alone.	P1	3	4.0	B-06 D-01
B-08	Shared playback and follow mode playback_sessions row, presenter claim / release, step sync through an API validated server-side, follower toggle, Explore and Rejoin, who-is-following indicator. Done when: a non-presenter calling the step endpoint is rejected and followers stay in sync within 500 ms.	P1	3	2.5	B-07 A-02
B-09	Canvas accessibility Text / table equivalent view, full keyboard stepping, reduced-motion mode, non-colour change markers, aria-live step announcements. Done when: a full trace can be stepped with keyboard and screen reader only.	P2	4	1.0	B-07
B-10	Ghost Driver voice explanation 'Explain this step' sends the selected runtime state to the AI route; shows text, optionally speaks via browser TTS, with mute / stop and captions. Done when: works with sound off and with sound on.	P2	4	1.0	B-07 C-01

Lane C - AI Tutor and Code Quality (18.5 blocks, 12 tasks)					
ID	Task and deliverable	Tier	Ph	Est	Needs
C-01	LLM gateway Provider adapter, server-only keys, zod-validated structured output with one retry, streaming, timeouts, response cache by hash, per-user rate limit, size limits, prompt-injection-safe templates (student code is delimited data). Done when: malformed model output never reaches the UI.	P0	1	1.5	00-03 A-01
C-04	Lint engine and adapters Analyzer interface; JS via ESLint Linter, Python via a stdlib-ast analyzer, C/C++/Java via compiler warnings plus heuristics; rules grouped as formatting, naming, smells, complexity, security hints; Diagnostic schema. Done when: each category fires on a planted sample in every language.	P0	1	3.0	00-03
C-02	Error explainer Context builder (file, language, NormalizedError, stdin), structured explanation {what, where, why, plain, fix, snippet, concepts}, Explain with AI button on failed runs, explanation card, line highlight, 'AI can be wrong' notice. Done when: a NameError run yields a beginner-level explanation pointing at the right line.	P0	2	2.0	C-01 B-04
C-03	Patch workflow Request a corrected version; Monaco diff preview; Accept / Reject; accept applies minimal edits to Y.Text (diff-match-patch) so other cursors survive; stale-base detection; decision logged. Also carries mentor-proposed edits. Done when: Reject leaves the document untouched and Accept syncs to everyone.	P0	2	2.5	C-01 A-04
C-05	Quality panel UI Monaco markers, Problems list grouped by category with filters, per-finding Explain (AI), maintainability summary (readability, structure, naming, duplication, potential bugs) with qualitative levels, 'feedback for learning - not a grade'. Done when: a finding is fixed and disappears on the next analysis.	P0	2	2.0	C-04 A-04
C-06	Hint mode (Socratic) Three-tier hint ladder (nudge, guiding question, near-answer), per-room default set by the mentor, hint tier logged. Done when: tier 1 and 2 never contain the corrected code.	P1	3	1.0	C-02
C-07	Code explainer Explain a selection, a function or the whole file; line-by-line walkthrough; short complexity note. Done when: selecting 5 lines and asking 'what does this do' returns a clear answer in under 6 s.	P1	3	1.0	C-01
C-08	AI code review Logic and structure review with the reason each suggestion is better, ordered by severity; any suggestion can be turned into a patch preview. Done when: review of the aliasing sample names the aliasing problem.	P1	3	1.0	C-03
C-09	Concept tagging and recurring-error detection Every explanation, lint finding and failed run is tagged with taxonomy concepts; the same category repeated inside a window sets a recurring flag that changes the prompt (different analogy). Done when: third identical mistake gets a visibly different explanation.	P1	3	1.0	C-02 D-03
C-10	AI evaluation set and demo cache 20 canned failing programs (5 languages x bug types) with expected key facts, a script that runs them and prints a rubric table, pre-baked cache for the demo programs. Done when: at least 17 of 20 explanations are judged correct by two teammates.	P1	3	1.0	C-02 B-05
C-11	Test-case suggestions and complexity explanation AI proposes edge-case inputs to try and a plain-language time/space note for the current code. Done when: suggested inputs can be pasted into stdin in one click.	P2	4	1.0	C-01
C-12	Practice problems and micro-lessons From a learner's weak concepts generate a short lesson and a practice task; answers checked by running them in the sandbox, not by the model. Done when: a flagged concept produces a lesson and a runnable exercise.	P2	4	1.5	C-01 D-04

Lane D - Learning, Classroom and Experience (21.0 blocks, 12 tasks)

ID	Task and deliverable	Tier	Ph	Est	Needs
D-01	Design system and app shell Tokens (light/dark), base components, workspace layout: file rail, editor, right dock (Video / Chat / AI / Quality / Progress), bottom panel (Console / Runs / Canvas). Empty, loading and error states; feature-flag switch for unfinished panels. Done when: shell renders every panel from fixtures.	P0	0	2.0	00-02
D-03	Learning events pipeline learning_events writer and emit helpers (run outcome, error category, AI help, hint tier, patch decision, lint fixed, checkpoint answer, heartbeat); concept taxonomy seed; error-to-concept rule map. Done when: a run produces one event row with the right concept tags.	P0	1	1.5	00-04 00-03
D-02	Permission-gated debugging Request / allow / deny / revoke / expire API, RLS extension so an active grantee can read the owner's runs, traces and explanations, realtime notifications, prompt modal, 'X is viewing' banner, revoke button, read-only mirror console, grantee actions: highlight line, propose edit, re-run (creates a new run owned by the grantee). Done when: revoke cuts access instantly and a third user never sees the data.	P0	2	3.0	B-03 C-03
D-04	Student progress view Run and success trend, error categories over time, concepts practised, observation sentences ('2 of 3 loop-boundary checks passed; retry recommended'), next-concept suggestions, 'what we record' panel. Done when: the seeded student shows believable observations without any score or rank.	P0	2	2.0	D-03
D-12	In-room chat LiveKit data-channel chat, share-selection-as-code-snippet message, unread badge. Done when: a pasted snippet renders highlighted and can be copied into the editor.	P0	2	1.0	A-07
D-05	Class trends (mentor) Concept x student matrix showing values and patterns (not colour alone), common errors chart, most-asked AI topics, active vs inactive, average debugging time; click through to a student's progress; no keystroke data. Done when: mentor sees '60% struggled with loop boundaries' from seed data.	P1	3	2.0	D-04
D-06	Predict and reflect checkpoints Presenter places a checkpoint on a trace step; each learner privately predicts (number / text / choice); answers hidden by RLS until reveal; live answered count; reveal shows actual state and the class distribution; each learner then gets an individual follow-up (AI proposes a variant, the sandbox computes the answer key); results become learning events. Done when: two learners give different predictions and both get personal follow-ups.	P1	3	4.0	B-06 B-07 B-08 D-03
D-07	Seed and demo data Demo class, three student personas with histories, the planted-bug programs, one-click 'reset demo'. Done when: reset returns the app to a known state in under 10 s.	P0	4	1.0	B-05 D-04
D-08	Landing page and onboarding Pitch landing page, join-with-code entry, guest mode, three-step first-run tour, plain-language role choice. Done when: a stranger joins a room from the landing page unaided.	P1	4	1.5	A-02
D-09	Stuck-detection nudge N failed runs of the same category within M minutes show a private prompt 'Ask for help?'; creates a help request visible in the mentor grid; never exposes the session. Done when: nudge appears on the 3rd repeat and is dismissible.	P2	4	0.5	A-09 D-03
D-10	AI session summary End-of-session recap per learner: what they tried, mistakes made, concepts touched, what to revisit. Done when: summary renders for a seeded session.	P2	4	1.0	D-03 C-01
D-11	Accessibility and UX pass Keyboard navigation across the app, focus rings, contrast check, aria labels, reduced motion, captions wherever audio exists, laptop and tablet layouts. Done when: the golden path is completable without a mouse.	P1	4	1.5	D-01

11. Risks and mitigations

Risk	L	I	Mitigation	Lane
Hosted Judge0 quota, latency or rate limits bite during the demo	M	H	Test 4 concurrent runs in T-00-05; limit runs per learner; show a Queued state; keep a second key or a small VM behind the same adapter	B
Render or Supabase cold start or pause	H	M	Keep-warm pings (T-A-12); warm-up step 10 minutes before the demo; consider a paid starter tier for the week	A
Tracer mishandles aliasing, deep recursion or large data	M	H	Caps, test corpus in T-B-06, fall back to plain output if a trace fails, keep demo programs small	B
LLM quality, latency or cost	M	H	Eval set (T-C-10), streaming, cache, hard spend cap, pre-baked demo answers, fast model by default	C
Venue Wi-Fi breaks video	M	M	Audio-only mode, phone hotspot, recorded backup video	A
AI patch collides with concurrent edits	L	M	Minimal diff applied as a Yjs transaction; stale-base check refreshes the preview	C
Scope creep: P1 work eats P0	H	H	Tiers, cut order in 10.3, feature flags, feature freeze at M4	Lead
Four lanes drift apart and integration fails late	M	H	Contracts first (T-00-03), fixtures, integration window each phase, main always deployable	All
Security slip: prompt injection through student code, leaked key, open WebSocket	L	H	Code as quoted data plus schema validation, server-only keys, JWT on WebSocket, RLS tests in T-00-04	A, C
Progress tracking looks like surveillance	L	M	Coarse events, 'what we record' panel, observation wording, no keystroke logging	D
Claude sessions do not share memory between teammates	H	M	CLAUDE.md, tracker files and docs in git; handoff note on every task	Lead
LiveKit free-tier limits	L	M	Check quota in T-00-05; small rooms; audio-only fallback	A

L = likelihood, I = impact (H high, M medium, L low).

12. Demo and submission plan

12.1 Golden path (about 7 minutes)

#	Time	What happens	Shows
1	0:00	Mentor creates room Loops-101 and shares the code; two students and a judge (guest) join in seconds	rooms, guest auth, presence
2	0:45	All edit one file; labelled cursors; mentor talks on video; someone shares a snippet in chat	CRDT, cursors, video, chat
3	1:30	Each learner runs the same file with different stdin; three independent outputs side by side	shared code, private execution
4	2:15	A student hits an <code>IndexError</code> ; Explain with AI; switch to Hint mode; then Patch, diff preview, Accept, code updates for everyone	AI tutor, hint ladder, safe patch
5	3:30	Quality panel flags nested loops, magic number, naming; explain one finding	quality analysis
6	4:00	Mentor grid shows a student stuck (status only); mentor requests access; student allows; mentor highlights a line; student revokes	permission-gated debugging
7	5:00	Trace the list-aliasing bug on the canvas; mentor presents, students follow; predict checkpoint, reveal, personal follow-up	canvas, shared playback, predict and reflect
8	6:15	Student progress view ('2 of 3 loop-boundary checks passed; retry recommended'); class trends	progress tracking
9	6:45	Close on the architecture figure and the roadmap list	P3 roadmap

12.2 Planted-bug programs (built in T-B-05, used by T-C-10 and T-D-07)

- Python: missing colon (`SyntaxError`) and an undefined variable (`NameError`)
- Python: loop off by one (`IndexError`) - the canvas and checkpoint example
- Python: `b = a` then mutate `b` (list aliasing) - the canvas signature example
- Python: recursion with no base case (`RecursionError`) - shows call frames stacking up
- Any language: infinite loop (`Timeout`)
- C: missing semicolon and undeclared variable; Java: wrong class name and missing import

12.3 Fallbacks

- Pre-baked AI answers for the planted programs so a slow model cannot stall the demo (T-C-10).
- Backup video of the full golden path and screenshots of every screen (T-X-04).
- Warm-up: ping Render, Supabase, Judge0, LiveKit and the LLM 10 minutes before you go on.
- Audio-only mode and a phone hotspot if venue Wi-Fi fails; a second laptop already signed in as the student.
- Rehearse one deliberate failure (slow AI or Wi-Fi drop) so the recovery is smooth.

12.4 Submission checklist

- README: what it is, architecture image, how to run it, live URL and demo credentials.
- Pitch material that matches what was actually built; move anything unbuilt onto a roadmap list.
- Optional evidence: 15-minute mini-pilot with 4-6 classmates (time to fix a bug with and without the AI explanation) so claims rest on data.
- Repository access for the judges if required.

13. Open decisions

13.1 Decisions

#	Decision	Options	Recommendation	By
D1	Time budget	Fix deadline, demo slot and hours per person	Write it down; apply the cut order in 10.3	Before phase 0
D2	Code execution backend	Hosted Judge0 or self-hosted on a small VM	Start hosted; move to a VM only if T-00-05 shows quota or latency trouble	T-00-05
D3	LLM provider and keys	Claude, Gemini, or another provider; who pays	Provider-agnostic gateway, fast and strong tiers, hard spend cap	T-00-01
D4	Identity	Accounts only, or accounts plus anonymous guest	Supabase Auth with Google and email link plus anonymous guest for judges	T-A-01
D5	Mentor visibility	Status only without grant, or nothing without grant	Status only, disclosed in the UI; content always needs a grant	T-D-02
D6	Yjs server	y-websocket or Hocuspocus	2-hour spike at the start of T-A-03; either meets the requirement	T-A-03
D7	Visualization languages	Python only, or more	Python only (compendium allows a Python-first subset); others show output only	T-B-06
D8	Lane owners	Self-select using section 9.2	Each lane gets an owner and a named backup	Before phase 0
D9	Repo and branching	GitHub private repo; trunk-based	As in 9.3	T-00-02
D10	Working mode	Parallel lanes, or single-machine relay	Parallel lanes; relay only as fallback (9.6)	Before phase 0
D11	Evidence for claims	Skip, or 15-minute mini-pilot	Do it if time remains after M3	T-X-05
D12	Tech stack	Recommended stack in 4.4, or your own choices per layer	Keep it unless a teammate already knows a different stack well; a familiar stack beats a perfect one	Before phase 0

Appendix A. Environment variables

Name	Where used	Secret?
NEXT_PUBLIC_SUPABASE_URL, NEXT_PUBLIC_SUPABASE_ANON_KEY	web	No (protected by RLS)
SUPABASE_SERVICE_ROLE_KEY	App API, collab service	Yes
APP_BASE_URL	App API (callback URL)	No
COLLAB_JWT_SECRET	App API, collab service	Yes
NEXT_PUBLIC_COLLAB_WS_URL	web	No
NEXT_PUBLIC_LIVEKIT_URL	web	No
LIVEKIT_API_KEY, LIVEKIT_API_SECRET	App API	Yes
JUDGE0_URL, JUDGE0_API_KEY	App API	Key: Yes
JUDGE0_CALLBACK_SECRET	App API (HMAC)	Yes
LLM_PROVIDER, LLM_MODEL_FAST, LLM_MODEL_STRONG	App API	No
LLM_API_KEY	App API	Yes
NEXT_PUBLIC_FF_* (feature flags)	web	No

Names only. Values go in the shared password manager and in Vercel / Render settings, never in git.

Appendix B. Judge0 reference

Language ids below are Judge0 CE 1.13.0 defaults. Newer builds add and renumber languages, so read GET /languages once at startup and map by name.

Language	File Judge0 uses	id (1.13.0)	Notes
Python 3	script.py	71	also runs the tracer and the Python analyzer
C (GCC)	main.c	50	compiler_options: -Wall -Wextra
C++ (GCC)	main.cpp	54	compiler_options: -Wall -Wextra
Java	Main.java	62	class must be named Main; rewrite on submit
JavaScript (Node)	script.js	63	-

Judge0 status	Name	Our category	Badge	Explain with AI
1, 2	In Queue, Processing	Running	blue	No
3	Accepted (finished without error; we send no expected output)	Success	green	No (Review offered)
5	Time Limit Exceeded	Timeout	amber	Yes (infinite-loop advice)
6	Compilation Error	Compilation Error	red	Yes
7 - 12	Runtime Error (SIGSEGV, SIGXFSZ, SIGFPE, SIGABRT, NZEC, other)	Runtime Error, or Memory Limit Exceeded when peak memory is at the limit	red	Yes
13, 14	Internal Error, Exec Format Error	Service Error	grey	No (offer retry)

Judge0 has no separate memory-limit status, so Memory Limit Exceeded is inferred. Status 4 (Wrong Answer) is never used because we do not grade.

Appendix C. Concept taxonomy

Seed list for the concept tags created in T-00-03 and used by T-D-03, T-C-09 and every progress view.

Concept	Meaning	Concept	Meaning
syntax-basics	colons, brackets, semicolons	lists-arrays	indexing, length
variables-types	variables and types	list-mutation-aliasing	two names, one list
input-output	reading input, printing	strings	strings and slicing
conditionals	if / else	dictionaries-maps	key / value
loops	for / while	exceptions-errors	reading and handling errors
loop-boundaries	off-by-one, range ends	classes-objects	OOP basics
functions-params	functions and parameters	pointers-memory	C and C++ memory
return-values	returning values	compilation-linking	C, C++, Java build errors
scope	local vs global	complexity-efficiency	nested loops, cost
recursion	base case, call stack	debugging-strategy	reproduce, isolate, fix

Appendix D. Definition of done

A task is DONE when:

1. Its 'Done when' line passes on the **deployed preview**, not only on localhost.
2. Types come from packages/shared; no new untyped escape hatches.
3. Loading, empty and error states exist.
4. No secret is in the client bundle or in git.
5. The tracker card is updated with a handoff note.
6. A teammate from an adjacent lane reviewed the PR and it is merged to main.
7. The golden path (section 12.1) still passes.

A feature is done when, in addition: it is reachable by keyboard, every control has a label, statuses carry text as well as colour, and the wording suits a beginner.

Appendix E. Glossary

Term	Meaning in this project
CRDT	Conflict-free replicated data type. Lets two people edit the same text at once and always converge, with no 'last save wins'. Yjs implements it.
Yjs, awareness	The CRDT library; awareness is its side channel for cursors, names and presence, which is not saved.
Monaco	The editor component from VS Code, used for the editor and the diff preview.
RLS	Row-level security. Rules inside Postgres that decide which rows each user may read or write, even if the API has a bug.
Realtime	Supabase feature that pushes database row changes to subscribed browsers, filtered by RLS.
SFU	Selective forwarding unit. The media server (LiveKit) that relays video and audio between participants.
Judge0	Open-source code execution service; runs one submission in an isolated sandbox with CPU, memory and time limits.
Lane	One person's area of ownership (A to D). Tasks are named T-lane-number.
Person-block	About two focused hours of one person; the unit of every estimate.
Golden path	The scripted demo route in section 12.1 that must always work on main.